# FROSTBIT: Compiling Boolean Filters over Compressed Bitmaps into Allocation-Free, Provably-Sized Execution Plans

William Liu
Exa
San Francisco, USA
wliu@exa.ai

## ABSTRACT

Roaring-format compressed bitmaps are the standard substrate for boolean filtering in search and analytics systems, and a decade of work has driven their per-container SIMD kernels to near the hardware ceiling. We show that in an end-to-end filtering engine the remaining performance no longer lives in the kernels but in the scaffolding around them: allocator traffic for every intermediate, representation decisions re-derived inside every operation, containers streamed that provably cannot reach the result, and fold orders chosen blind to the memory hierarchy.

FROSTBIT eliminates the scaffolding by construction. A boolean filter expression is compiled, once, into a complete execution *manifest*: a bottom-up analysis derives every operator's exact output-container set with a *proven byte ceiling per container* (so the execution path contains no allocation or growth code at all), fixes every representation and algorithm choice through calibrated cost models — including a fan-in-aware array→bitmap promotion rule and a fold-order decision made from the plan's own memory-footprint bound — and attaches a container-granular liveness mask so that every 64 Ki-value block that cannot survive the expression root is skipped before a byte of it is streamed. The executor replays this manifest through per-thread buffer pools and serializes results *in place* inside its working arena: in steady state a query performs zero heap allocations, and we bound its peak working memory in closed form from the plan.

Against the standard Rust Roaring library, FROSTBIT wins every cell of a 2–16-way operation sweep by 1.2–15.5× (and all but two noise-level cells against Roaring's nightly SIMD build), evaluates a 25,000-tree filter corpus 3.4× (2.25×) faster end-to-end, prunes selective filters up to 33×, and short-circuits contradictory filters by 2,900×. FROSTBIT is the production filtering substrate of Exa, a web-scale search engine, where every boolean filter predicate evaluates through it.

## 1 INTRODUCTION

Boolean filtering — "documents matching $d \in domains \wedge \ell = en \wedge t \in [t_0, t_1)$" — sits on the critical path of every modern retrieval and analytics system. The dominant physical design is by now uncontroversial: per-attribute compressed bitmaps in the two-level Roaring layout [10, 12], whose 64 Ki-value *containers* adaptively choose among sorted u16 arrays, 8 KiB packed bitmaps, and run lists, and whose per-container kernels have been the subject of a long and effective line of SIMD work [27, 37, 38, 41, 43].

Operating such a system in production at Exa led us to a diagnosis that this paper develops into a design: *the kernels are no longer the bottleneck; the glue is.* Consider what a conventional engine does to evaluate an $n$-ary filter tree. It allocates a fresh result structure for every intermediate node (in our profiles of the Rust Roaring library, roughly 6% of end-to-end runtime is allocator time before any tail effects); at every container it re-derives the same representation questions the previous operator already answered; it faithfully streams, decodes, and merges containers whose keys a one-line argument shows cannot survive the enclosing conjunction; and it visits its operands in an order fixed by the expression, oblivious to whether that order thrashes L1 or streams sequentially.

FROSTBIT's design premise is that every one of those costs is a *planning failure*, not an execution necessity. Filters in serving systems are compiled once and evaluated many times against immutable index segments; the index is frozen by construction. Under those conditions an analysis pass can legitimately pre-compute *everything*:

- the exact set of output containers of every operator, each with a byte ceiling proven sufficient for every intermediate state its fold can reach (§4.2) — making allocation-free execution an *invariant*, not an aspiration;
- every algorithmic decision, as explicit cost models over quantities the plan already knows: kernel tier per container pair, array→bitmap promotion as a function of per-key *fan-in and* cardinality, and — to our knowledge new — the fold order of each operator, selected by comparing the plan's proven accumulator footprint to the cache (§5–§6);
- a container-granular liveness mask and compiled short-circuit guards, so the executor streams a provably sufficient *subset* of the input containers — dead blocks are never touched (§7);
- all working memory, drawn from per-thread pools and bounded in closed form (§8); results serialize *in place*, so the fold's writes are the output's bytes.

The executor that remains is deliberately boring: a straight-line replay of the manifest. We quantify what the boring executor is worth in §9: starting from kernels at parity with the state of the art (verified against a 310-paper survey of this literature), the scaffolding elimination is the difference between parity and winning every measured cell.

**Contributions.** (1) The plan-as-manifest architecture with proven per-container capacity clamps and a closed-form working-memory bound (§4–§8). (2) Calibrated cost-model dispatch at three levels, including fan-in-aware promotion and cache-footprint-driven fold-order selection, each with the measured constants and break-even derivations (§5–§6). (3) Automatic container-granular pruning across

boolean expression trees, with a soundness argument and a streamed-bytes account (§7). (4) An evaluation with complete per-technique ablations *including negative results*, a 25,000-tree corpus, and a per-tree audit methodology (§9); plus production deployment experience (§10).

## 2 BACKGROUND AND RELATED WORK

We position FROSTBIT against four bodies of work, surveyed systematically during its design (310 collected papers; the corpus and index accompany the artifact).

**Compressed bitmap indexes.** The word-aligned RLE family — BBC [1], WAH [6], PLWAH [7], CONCISE [8], and successors — optimizes space and bulk boolean throughput but sacrifices random access; sorting heuristics amplify run lengths [9]. Roaring [10–12] restructured the field around a two-level design with per-container adaptive representations, and is the baseline we compare against and remain wire-compatible with. Bitmap index theory — design spaces and encodings — goes back to variant indexes [3] and formal design studies [4]; bit-sliced arithmetic [5] and bitmapped join indices [2] are complementary encodings above the container layer. Updatable designs (UpBit [13], TEB [14] and its in-place updates [15], CUBIT [16]) solve a problem FROSTBIT deliberately excludes: our bitmaps are frozen artifacts of an indexing pipeline, and that immutability is precisely what makes whole-plan static analysis sound. The bitmap-versus-inverted-list boundary is mapped empirically in [17].

**Postings compression.** The inverted-index compression literature — interpolative coding [18], word-aligned codes [19], PForDelta [20, 21], vectorized codecs [22, 25, 29], Elias–Fano [23, 24, 28], document reordering [26, 30], surveyed in [31] — optimizes a different point: sequential decode of $d$-gapped lists. FROSTBIT targets the regime Roaring occupies — operate-in-compressed-form with random access — but the postings literature's lesson that *layout decisions should be made by optimization, not convention* (e.g., partitioned Elias–Fano [24]) is the same philosophy FROSTBIT applies to execution planning.

**Set intersection algorithms.** Adaptive and instance-optimal intersection begins with [32] and the Barbay line [33, 34]; engineering studies for inverted indexes include [35, 36, 42]. The SIMD lineage — shuffle-based compare-all-pairs [27, 37], branch-reduced merge networks [38], QFilter [41], FESIA's segment-bitmap prefilters [43], and VP2INTERSECT alternatives [44] — defines today's kernel frontier; worst-case-optimal join systems embed the same kernels at relational scope [39, 40]. FROSTBIT's kernels are a portable synthesis of this line (§5); the contribution is not a new kernel but the compile-time selection layer above the known ones — closest in spirit to the query-time cost models of [42], but static, container-granular, and coupled to capacity proofs.

**Bitmaps and signatures in systems.** Bitmap and signature machinery pervades serving systems: classical signature files [45, 46], FastBit in scientific data [47], BitFunnel's bit-sliced signatures in the Bing search fleet [56], bitmap filtering inside analytics engines (Druid [52, 54], Pinot [58], Procella [59], ClickHouse [60], Dremel [48]), social-graph retrieval [49], and bit-parallel scan accelerators (BitWeaving [50], ByteSlice [53], Column Imprints [51], Column Sketches [57], Data Blocks [55]). These papers document container structures and scan layouts; none, to our knowledge, documents compiling boolean filter *expressions* into statically-proven execution plans. That layer — the one this paper is about — appears to be tribal knowledge at best, and its absence from the literature is corroborated by the absence of any publication for the several industrial bitmap engines (Pilosa, Kylin-class systems) our survey sought.

## 3 PRELIMINARIES AND NOTATION

A FROSTBIT bitmap represents $S \subseteq U = [0, 2^{32})$. Key $\kappa(x) = \lfloor x/2^{16} \rfloor$ partitions $S$ into containers $c_k = \{x \bmod 2^{16} : x \in S, \kappa(x) = k\}$, stored as one of: a sorted `u16` *array* ($|c| \le A_{\max} = 4096$), a packed *bitmap* of $\mathsf{B} = 8192$ bytes, or a *run* list of $\rho(c)$ intervals ($4\rho + 2$ bytes, $\rho \le \rho_{\max} = 2047$). The stored size of a container is

$$\text{stored}(c) = \begin{cases} 2\,|c| & \text{array} \\ \mathsf{B} & \text{bitmap} \\ 2 + 4\rho(c) & \text{runs.} \end{cases}$$

The on-disk format is a 16-byte header, a structure-of-arrays directory (8 bytes per container), and 64-byte-aligned payloads; buffers are validated once and thereafter read zero-copy, including from `mmap`.

A *filter expression* is a tree $T$ over leaves (frozen bitmaps) with operators $\wedge, \vee, \setminus$ of arbitrary arity (binary for $\setminus$). Same-operator edges are flattened at construction, so an operator's *fan-in* $n$ is its flattened operand count. We write $K(v)$ for the set of container keys node $v$ can emit and $\text{card}_k(v)$, $\rho_k(v)$ for per-key bounds.

## 4 THE ANALYSIS PASS

Constructing an expression *is* its analysis: each combinator fuses its children's plans into a flat post-order program and computes, bottom up, the three artifacts below. Plans are built once and replayed per evaluation, matching the compiled-filter usage pattern of serving systems.

### 4.1 Shapes

DEFINITION 1 (SHAPE). *The shape $\mathcal{S}(v)$ of node $v$ is a key-sorted sequence of records $(k, \widehat{\text{card}}_k, \widehat{\rho}_k, \widehat{\tau}_k)$ giving, for every key $v$ can emit, upper bounds on the output container's cardinality and run count and its representation under the kernel ladder.*

Leaf shapes are exact (read from the directory). Interior shapes follow the operator semantics with interval arithmetic: $K(\wedge) = \bigcap_i K(v_i)$ with $\widehat{\text{card}}_k = \min_i \widehat{\text{card}}_k(v_i)$; $K(\vee) = \bigcup_i K(v_i)$ with $\widehat{\text{card}}_k = \sum_i \widehat{\text{card}}_k(v_i)$ (saturating), and $K(\setminus) = K(v_{\text{lhs}})$ with lhs bounds. Shapes are *sound over-approximations*: $K$ contains every key the executor can emit and every bound dominates the runtime value. Soundness is compositional and survives flattening because each spliced child carries its *operand weight* (its flattened arity) into the parent's analysis, so per-key fan-in bounds $\widehat{n}_k$ are also upper bounds — an accounting subtlety we initially got wrong (counting tree children rather than flattened operands), caught immediately by the capacity stress suite of §4.2.

## 4.2 Capacity clamps

From its shape, every operator receives an *arena plan*: one output slot per key of $K$, with byte capacity

$$\mathrm{cap}(k) \;=\; \begin{cases} \max\big(\mathrm{stored}_{\mathrm{lhs}}(k),\ \sigma(\widehat{\mathrm{card}}_k)\big) & \setminus \\ \sigma(\widehat{\mathrm{card}}_k) & \wedge \\ \max\big(2\widehat{\mathrm{card}}_k,\ 2{+}4\widehat{\rho}_k,\ \max_i \mathrm{stored}_i(k)\big) \text{ or } \mathsf{B} & \vee \end{cases}$$

where $\sigma(m) = 2m$ if $m \le A_{\max}$, else $\mathsf{B}$ (the *shrink bound*), and the union case takes $\mathsf{B}$ iff the plan decides the key accumulates as a bitmap or native run form (§5).

Lemma 1 (Clamp soundness). *For every operator, key, and input instance consistent with the shapes, every intermediate state of the fold at key $k$ occupies at most $\mathrm{cap}(k)$ bytes.*

Proof sketch. Case analysis on the kernel ladder. ($\wedge$) The accumulator seeds from the per-key minimum-cardinality operand and only shrinks; if $\widehat{\mathrm{card}}_k \le A_{\max}$ every state is an array of at most $2\widehat{\mathrm{card}}_k$ bytes, otherwise states are runs or bitmaps, both $\le \mathsf{B} = \sigma$. ($\setminus$) A key untouched by any subtrahend is a verbatim copy ($\mathrm{stored}_{\mathrm{lhs}}$); a touched key only shrinks from the lhs ($\sigma$); run-splitting states are bounded by $2{+}4(\rho_{\mathrm{lhs}}{+}\mathit{items}_{\mathrm{rhs}})$ bytes and occur only under a $\mathsf{B}$-clamp, with conversion to bitmap forced before $\rho$ can exceed $\rho_{\max}$. ($\vee$) Array accumulation occurs only when the plan proved $\sum_i \widehat{\mathrm{card}}$ fits an array (every prefix then fits); otherwise the clamp is $\mathsf{B}$ and all of bitmap, run ($\rho \le \rho_{\max}$), and conversion states fit. The executor never re-derives these decisions — *the clamp is the decision* — so planner/kernel divergence is structurally impossible. □

Invariant 1 (Allocation-freeness). *The execution path contains no allocation, growth, or reallocation code.*

This is not an optimization applied to a fast path; the code path does not exist. Debug builds assert every recorded container against its clamp, and an 11,600-case randomized stress suite exercises the proof obligations of Lemma 1 (it caught, e.g., the verbatim-copy case a max was originally missing from).

## 4.3 Arena layout and the memory bound

An operator with $n_s$ planned slots occupies one contiguous arena: reserved output front $H(n_s) = \mathrm{align}_{64}(16 + 8n_s)$, the slot region, an optional mirror region (§6), and fixed scratch $2\mathsf{B}$:

$$A(P) \;=\; H(n_s) \;+\; (1{+}\delta_P)\,\mathrm{align}_{64}\Big(\sum_{k\in K} \mathrm{align}_8(\mathrm{cap}(k))\Big) \;+\; 2\mathsf{B}, \quad (1)$$

with $\delta_P \in \{0,1\}$ the double-buffering bit. Since $\mathrm{cap} \le \mathsf{B}$ always, $A(P) \le H(n_s) + (1{+}\delta_P)\,n_s\mathsf{B} + 2\mathsf{B}$; and because $\delta_P = 1$ only when $\sum \mathrm{cap} \le F_{\max} = 64\,\mathrm{KiB}$ (§6), the mirror adds at most 64 KiB. For a tree, arenas live on the evaluation stack, so peak working memory is bounded by the plan-time quantity $\max_{\text{stack states } s} \sum_{v\in s} A(P_v)$ — computable at analysis time, and in practice dominated by the widest single operator ($\le n_s\mathsf{B}$+const, i.e. ~8 KiB per distinct output key).

**Table 1: Calibrated kernel constants (Apple M-series, medians).**

| | | |
|---|---|---|
| $c_b$ | 8-lane merge block (compare-all-pairs) | 1.45 ns |
| $c_m$ | per element streamed through an array merge | 0.85 ns |
| $c_s$ | per value scattered into a bitmap | 1.2 ns |
| $c_0$ | bitmap clear + popcount (fixed) | ≈ 450 ns |
| $c_p$ | bitmap membership probe (hoisted loop) | ≈ 1 ns |

## 4.4 In-place serialization

Because $H(n_s)$ was reserved up front and slots were laid out with cap ≥ payload, serialization compacts payloads leftward inside the arena and emits the arena's own buffer as the result.

Lemma 2 (Leftward compaction). *Writing outputs in key order at $d_i = \mathrm{base} + \sum_{j<i} \mathrm{size}_j$ (with per-type alignment) never overwrites an unread source: $d_i \le s_i$ for all $i$.*

Proof sketch. Sources sit at $s_i = H + \sum_{j<i} \mathrm{align}_8(\mathrm{cap}_j) + \Delta_i \ge \mathrm{base} + \sum_{j<i} \mathrm{size}_j = d_i$, since $\mathrm{base} \le H$ and $\mathrm{size}_j \le \mathrm{cap}_j$ pointwise; mirror-side sources lie beyond the entire primary region, strengthening the inequality. Alignment rounding preserves it because sources are at least as aligned as destinations. □

The practical content of Lemma 2 is a whole pass of memory traffic *removed*: a conventional engine copies every output byte from working memory into a result allocation. Profiling located this precisely — FROSTBIT's 16-way difference fold already beat Roaring (185 vs. 190 *ns*) while the end-to-end operation lost (206 vs. 190 *ns*); the entire deficit was the output copy. Eliminating it turned every large-result cell around (§9.3).

# 5 COST-MODEL DISPATCH

Execution decisions are functions of plan-known quantities, with constants calibrated once per microarchitecture (Table 1) and every crossover ablated (§9.3).

**Kernel tiers.** For an array pair $|A| \le |B|$: balanced pairs ($|B| \le 4|A|$) run a shuffle-merge at $c_b\,\lceil(|A|{+}|B|)/8\rceil$ — compare-all-pairs over rotations of *both* operands (5 permutations per block rather than 7) with branchless table compaction; heavy skew runs galloping binary search at $\approx c_g|A|\log_2|B|$, taken only under the margin test $8\,|A|\log_2|B| < |B|$, i.e. only when it beats the middle tier's linear window advance by 8× — binary search mispredicts, so paper parity is insufficient; the middle tier is a galloping broadcast-scan. Every tier is reached by data observed in production shapes, and the two boundaries were set empirically (the cost-model gallop gate replaced a fixed ratio that regressed five-way conjunctions by 26%).

**Fan-in-aware promotion.** A union key accumulating as an array costs each merge a re-stream of the accumulator; with fan-in $n$ and summed cardinality $s$ (even splits),

$$C_{\mathrm{arr}}(n,s) \approx c_m\, s\Big(\tfrac{n+1}{2} - \tfrac{1}{n}\Big), \qquad C_{\mathrm{bmp}}(s) \approx c_0 + c_s\, s. \quad (2)$$

Solving $C_{\mathrm{arr}} > C_{\mathrm{bmp}}$ with Table 1 yields no crossover at $n \le 3$ for any $s \le A_{\max}$, and thresholds $s \gtrsim 980, 480, 200$ at $n = 4, 5, 8$; we encode it as $\textsc{promote}(n,s) \equiv n \ge 4 \wedge s\,(n-3) > 1000$. The classical fixed threshold (promote at $s > 256$ regardless of $n$, as

in CRoaring-derived engines) is dominated: tuned low it converts the OR-groups of low-fan-in conjunctions into bitmaps their parent consumes more slowly (measured +29% on that shape); tuned high it strands wide unions in array form (measured 3.7× on 4-way unions, Table 4). Fan-in bounds come from the weighted shapes of §4.1; the clamp is the single authority at execution.

# 6 FOLD ORDER AND THE MEMORY HIERARCHY

An $n$-ary fold traverses keys × operands. *Key-major* order completes each key before the next: one accumulator stays register/L1 resident, but each of the $n$ operand streams is resumed once per key — $nK$ stream resumptions across $K$ keys, each restarting the prefetcher. *Partner-major* order streams operand $j$ end-to-end against all accumulators before operand $j$+1: perfectly sequential input traffic, but the entire accumulator set is revisited $n$ times.

Both orders move the same bytes; they differ in *arrangement*, and each is optimal in a regime the plan can identify. Profiling 16-way differences located a +50% per-block anomaly entirely in memory: the merge kernel ran at 1.45 ns/block standalone and 1.56 ns in a single-key control, but 2.17 ns inside the 32-key key-major fold — interleaved resumption of 16 streams defeating L1 — while the identical workload under partner-major recovered the standalone rate. Dense unions then exhibited the mirror image: with per-key bitmap accumulators the accumulator set is $K \cdot \mathsf{B}$ (128 KiB at $K$=16), revisiting it each pass thrashes L1, and key-major won by 12%.

The plan already holds the discriminating quantity: the accumulator footprint $F(P) = \sum_k \mathrm{cap}(k)$ of Eq. 1. FROSTBIT folds partner-major iff $F(P) \le F_{\max} = 64\,\mathrm{KiB}$ (half of L1D, leaving room for one streaming operand and the mirror side), allocating the mirror region ($\delta_P$=1) so array merges alternate slot sides instead of staging through scratch. Intersection is the measured exception: a key-count scaling control (16-way AND, 1 vs. 32 keys: 695 ns → 21.6 $ns$, 31.1× for 32× keys) shows no super-linear term — the accumulator collapses after the first fold and partners are galloped into, not re-streamed — and partner-major seeding would require clamping by the *driver's* cardinality rather than the per-key minimum, weakening Lemma 1. Each operator thus runs the order measurement assigned it: difference and union dispatch on $F(P)$; intersection stays key-major, by evidence rather than default.

# 7 STREAMING THE PROVABLE MINIMUM

The manifest eliminates work at three granularities. Throughout, "container granularity" is Roaring's own: 64 Ki-value blocks.

## 7.1 Hole-punching

DEFINITION 2 (LIVENESS MASK). $\mathcal{M}(T) = K(\mathrm{root}(T))$ *under the shape recurrences: intersection of child key sets at* ∧*, union at* ∨*, left child at* \.

LEMMA 3 (PRUNING SOUNDNESS). *For any leaf* $\ell$ *and key* $k \notin \mathcal{M}(T)$*, removing container* $c_k(\ell)$ *from the input changes no output value. Pruning is monotone:* $\mathcal{M}$ *over-approximates the true surviving key set, so no contributing container is ever dropped.*

PROOF SKETCH. By induction on $T$: a key absent from $K(v)$ yields the empty container at $v$ under all three operators, and empty containers are identities (for ∨) or annihilators (for ∧; left-absorbing for \) that propagate to the root. Since the executor's keys are a subset of shape keys (soundness, §4.1), masking to $\mathcal{M}(T)$ preserves every output. □

The planner derives $\mathcal{M}$ automatically whenever $|K(\mathrm{root})| < \max_i |K(v_i)|$ — i.e., exactly when pruning is guaranteed non-trivial — materializes it as an 8 KiB bitset, and every leaf cursor skips dead keys in place.

**Streamed-bytes account.** Without the mask, evaluating $T$ streams $\sum_\ell \sum_{k \in K(\ell)} \mathrm{stored}(c_k(\ell))$ input bytes; with it, keys outside $\mathcal{M}(T)$ contribute zero:

$$\Delta_{\mathrm{stream}} = \sum_{\ell \in \mathrm{leaves}(T)} \sum_{k \in K(\ell) \setminus \mathcal{M}(T)} \mathrm{stored}\big(c_k(\ell)\big). \tag{3}$$

In the selective-filter benchmark (one 4-key conjunct against four 256-key disjunction legs) this removes ∼ 98% of input bytes and yields 7.35 $ns$ vs. 241 $ns$ unpruned (33×) and 374 $ns$ for Roaring's SIMD build; across the 25k-tree corpus, enabling automatic derivation was worth −15.3% end-to-end ($p < 0.05$). We believe this is the first published container-granular analogue of zone-map pruning that acts *across* a boolean expression tree rather than along a scan.

## 7.2 Compiled short-circuits

The program carries *guard* instructions after every non-flattened ∧-operand and every \-minuend: if the operand just produced is empty, the guard pops the partial operands and jumps past the fold — sibling subtrees are never evaluated. Guards encode *relative* jump offsets so they survive splicing into enclosing plans. A contradictory filter (empty subtree beside an expensive disjunction) evaluates in 133 ns against 392 $ns$ for Roaring (∼ 2900×): the disjunction is simply never run. Within operators, intersection fuses population count into the AND and abandons a key the moment it empties; partners fold most-selective-first; partner-major passes skip dead slots.

# 8 MEMORY DISCIPLINE

Four per-thread pools back execution: the op arena (Eq. 1), fold cursor/reference scratch, the evaluator's operand stack, and the result buffers themselves. Pools hand out lifetime-erased buffers that are only ever stored empty; arenas chain as operator inputs without serialization until the root; and in-place serialization (Lemma 2) swaps the arena's buffer out as the result while a pooled result buffer rotates back — the pools circulate, so *steady-state evaluation performs zero heap allocations*, a property we verify by sampling profiles (the allocator does not appear) rather than assert. First use on a thread allocates once and joins the cycle; there is no pre-warm API to misuse.

What this removes, quantitatively: the baseline's allocator time (≈ 6% of Roaring's profile in our workloads — a fresh container vector per intermediate operation); the output copy (§4.4), worth 2–9 $ns$ on sparse 8–16-way operations and up to 30% on dense 2-way ones; and the allocator-induced tail variance that motivated the design in production (§10).

**Table 2: 8-way folds, medians in µs (sparse arrays / dense bitmaps / run containers). R = Roaring default, $R^s$ = nightly SIMD build.**

| op | FROSTBIT | R | $R^s$ |
|---|---|---|---|
| ∧ | **17** / **12** / **2.7** | 73 / 23 / 4.6 | 25 / 22 / 4.4 |
| ∨ | **117** / **21** / **3.0** | 1200 / 39 / 7.1 | 789 / 39 / 7.2 |
| \ | **72** / **53** / **2.0** | 1110 / 158 / 5.0 | 80 / 155 / 5.1 |

**Table 3: Filter-tree shapes, medians (µs), plan construction included.**

| shape | FROSTBIT | R | $R^s$ |
|---|---|---|---|
| conj5 (nested ANDs, flattens 5-way) | **34.1** | 315 | 80.3 |
| filter (base ∧ OR ∧ ¬lang) | **17.5** | 41.0 | 43.7 |
| dnf (OR of AND-groups) | **51.8** | 582 | 78.6 |
| cnf3 (AND of OR-groups) | **72.6** | 331 | 102 |
| 25k corpus (s) | **2.63** | 9.06 | 5.92 |
| selective (hole-punched) | **7.35** | 846 | 374 |
| contradiction (guarded) | **0.133** | 820 | 392 |

# 9 EVALUATION

**Setup.** Rust 1.8x / nightly for the SIMD baseline; Apple M-series (aarch64, NEON; x86 SSE ports compile and are CI-targeted but all numbers here are aarch64); Criterion medians, 30 samples × 4 s (corpus: 10 × 20 s). Baselines: Rust `roaring` 0.11.4 in *both* of its configurations — default (what stable-toolchain users run) and the nightly-only `simd` feature (its strongest form) — via a two-pass runner into one report. All FROSTBIT tree numbers include plan construction. Correctness: differential suites against Roaring (random trees, punched trees, run containers, 10M-element round trips) plus the 11.6k-case clamp stress; all evaluation configurations pass all suites.

## 9.1 Operation sweep

Table 2 reports the 8-way column of a 2–16-way sweep over three container regimes. Against default Roaring, FROSTBIT wins *every cell of the full sweep* by 1.2–15.5× (sparse-array ops by 3.6–15.5×: Roaring's array kernels are scalar without the nightly feature; FROSTBIT's SIMD always runs). Against the SIMD build, FROSTBIT wins every cell except two 16-way cells at noise level (0.98×, 0.99×). Union at 4-way sparse — the fan-in promotion's home regime — runs 54.9 $ns$ vs. 517.6/277.3 $ns$.

## 9.2 Filter-tree corpus

The end-to-end benchmark is 25,000 deterministic random filter trees (295,455 leaves; log-spread sizes 2–100 leaves, depths 2–15; per-tree shape profiles from deep AND-chains to flat 100-way ORs; 100 trees pinned to both extremes), each built, analyzed, and materialized per iteration. FROSTBIT: **2.63 s** (9.5K trees/s); Roaring default: 9.06 s (3.45×); Roaring SIMD: 5.92 s (2.25×). Named shapes (Table 3) bracket the production patterns.

**Table 4: Ablation ledger (kept mechanisms above the line, rejected below; all numbers measured).**

| mechanism | effect |
|---|---|
| balanced shuffle-merge (vs. scan) | dnf 130 → 57 $ns$ |
| register-resident merge rewrite | 2-way AND 49 → 24.8 $ns$ |
| fan-in promotion | 4-way OR 203 → 54.9 $ns$; guards intact |
| partner-major \ (footprint-gated) | 8-way sparse 85 → 72 $ns$ |
| in-place serialization | dense 2-way OR 7.2 → 5.1 $ns$ |
| hoisted bitmap probe | audit offenders 1245 → 743 |
| automatic hole-punching | corpus 3.10 → 2.63 s |
| branchless ∧ emit (gated) | dense-hit emit 2.07 → 0.95 $ns$ |
| eager bitmap intermediates ($s$>256) | cnf3 89 → 115 $ns$: rejected |
| branchless scalar ∨ merge | 72 → 91 $ns$: rejected |
| deferred vector-mask accumulation | \ 83 → 97 $ns$: rejected |
| window pre-clipping | redundant w/ gallop: rejected |
| partner-major ∧ | linear-scaling control: no gain |

## 9.3 Ablations, including the failures

Every mechanism entered under a keep-only-if-faster rule (Table 4); the rejected designs bound the space as sharply as the accepted ones. Three deserve emphasis. (i) The in-place-serialization ablation is a case study in profiling the right layer: the fold beat the baseline while the operation lost, and the gap was exactly the output copy. (ii) The branch study: an adversarial-vs-predictable input sweep showed the merge advance branches cost *zero* (predictable-pattern 1.43 $ns$ vs. adversarial 1.45 $ns$ at identical work), a branchless rewrite of the scalar union merge *regressed* 26%, yet branchless table compaction of the intersection emit won 2.2× at dense overlap — branchless transformations pay only where decisions are dense *and* data-dependent, and each direction is worth measuring. (iii) The per-tree audit loop — time all 25k trees on both engines, explain the worst offender structurally, fix the class — drove two mechanisms into the system (the hoisted bitmap probe, automatic hole-punching) and is, we would argue, a transferable methodology.

## 9.4 Positioning against the kernel literature

Because FROSTBIT's claims live above the kernels, we verified the kernels themselves are not the story: our balanced merge matches the compare-all-pairs family [27, 37] with the both-operand rotation of [44]; our union network is Inoue-style [38]; our skew tiers subsume the galloping tradition [32, 35]. A per-block throughput comparison against the Rust Roaring SIMD kernels shows parity to +16% in FROSTBIT's favor at equal work — consistent with our thesis that the end-to-end gaps of Tables 2–3 are scaffolding, not kernels.

# 10 DEPLOYMENT AT EXA

FROSTBIT is the foundational bitmap structure for *all* boolean filtering at Exa, a production web-scale search engine: every filter predicate — domain, language, time, category, and arbitrary boolean combinations — compiles to a FROSTBIT expression evaluated against frozen per-segment bitmaps served from memory maps. Three properties carry the operational weight. Plans compile once per query

and replay across segments, so analysis cost amortizes exactly as §4 assumes. The zero-allocation invariant removes the allocator from tail latency entirely — the original production complaint that motivated the design. And hole-punching converts the dominant production pattern (one narrow conjunct beside wide disjunctions) from the engine's worst case into its best. The frozen-segment model is not a restriction we tolerate but the assumption that makes the whole analysis sound; updates arrive as new segments from the indexing pipeline, as in any LSM-style search architecture.

## 11 CONCLUSION

After a decade of kernel optimization, the residual 1.2–15× in boolean bitmap filtering was hiding in the scaffolding. FROSTBIT reclaims it with an analysis pass strong enough to make execution trivial: every output pre-sized under a proven clamp, every algorithm and fold order chosen by a calibrated model over plan-known quantities, every dead container pruned before it is streamed, and every byte of working memory pooled and bounded in closed form — so the executor just replays a proof. The library, differential suites, 25k-tree corpus, audit tooling, and the 310-paper survey index are open source.

## REFERENCES


[1] G. Antoshenkov. Byte-aligned bitmap compression. *DCC*, 1995.
[2] P. O'Neil, G. Graefe. Multi-table joins through bitmapped join indices. *SIGMOD Record* 24(3), 1995.
[3] P. O'Neil, D. Quass. Improved query performance with variant indexes. *SIGMOD*, 1997.
[4] C.-Y. Chan, Y. E. Ioannidis. Bitmap index design and evaluation. *SIGMOD*, 1998.
[5] D. Rinfret, P. O'Neil, E. O'Neil. Bit-sliced index arithmetic. *SIGMOD*, 2001.
[6] K. Wu, E. J. Otoo, A. Shoshani. Optimizing bitmap indices with efficient compression. *ACM TODS* 31(1), 2006.
[7] F. Deliège, T. B. Pedersen. Position list word aligned hybrid: optimizing space and performance for compressed bitmaps. *EDBT*, 2010.
[8] A. Colantonio, R. Di Pietro. CONCISE: compressed 'n' composable integer set. *Inf. Process. Lett.* 110(16), 2010.
[9] D. Lemire, O. Kaser, K. Aouiche. Sorting improves word-aligned bitmap indexes. *Data & Knowl. Eng.* 69(1), 2010.
[10] S. Chambi, D. Lemire, O. Kaser, R. Godin. Better bitmap performance with Roaring bitmaps. *Softw. Pract. Exp.* 46(5), 2016.
[11] D. Lemire, G. Ssi-Yan-Kai, O. Kaser. Consistently faster and smaller compressed bitmaps with Roaring. *Softw. Pract. Exp.* 46(11), 2016.
[12] D. Lemire, O. Kaser, N. Kurz, L. Deri, C. O'Hara, F. Saint-Jacques, G. Ssi-Yan-Kai. Roaring bitmaps: implementation of an optimized software library. *Softw. Pract. Exp.* 48(4), 2018.
[13] M. Athanassoulis, Z. Yan, S. Idreos. UpBit: scalable in-memory updatable bitmap indexing. *SIGMOD*, 2016.
[14] H. Lang, A. Beischl, V. Leis, P. Boncz, T. Neumann, A. Kemper. Tree-encoded bitmaps. *SIGMOD*, 2020.
[15] M. P. Saputra, E. Tzirita Zacharatou, S. Chatzimilioudis et al. In-place updates in tree-encoded bitmaps. *DaMoN*, 2022.
[16] J. Wang, M. Athanassoulis. CUBIT: concurrent updatable bitmap indexing. *PVLDB* 17, 2024.
[17] J. Wang, C. Lin, Y. Papakonstantinou, S. Swanson. An experimental study of bitmap compression vs. inverted list compression. *SIGMOD*, 2017.
[18] A. Moffat, L. Stuiver. Binary interpolative coding for effective index compression. *Inf. Retrieval* 3(1), 2000.
[19] V. N. Anh, A. Moffat. Inverted index compression using word-aligned binary codes. *Inf. Retrieval* 8(1), 2005.
[20] M. Zukowski, S. Héman, N. Nes, P. Boncz. Super-scalar RAM-CPU cache compression. *ICDE*, 2006.
[21] H. Yan, S. Ding, T. Suel. Inverted index compression and query processing with optimized document ordering. *WWW*, 2009.
[22] B. Schlegel, R. Gemulla, W. Lehner. Fast integer compression using SIMD instructions. *DaMoN*, 2010.
[23] S. Vigna. Quasi-succinct indices. *WSDM*, 2013.
[24] G. Ottaviano, R. Venturini. Partitioned Elias-Fano indexes. *SIGIR*, 2014.
[25] D. Lemire, L. Boytsov. Decoding billions of integers per second through vectorization. *Softw. Pract. Exp.* 45(1), 2015.
[26] L. Dhulipala, I. Kabiljo, B. Karrer, G. Ottaviano, S. Pupyrev, A. Shalita. Compressing graphs and indexes with recursive graph bisection. *KDD*, 2016.
[27] D. Lemire, L. Boytsov, N. Kurz. SIMD compression and the intersection of sorted integers. *Softw. Pract. Exp.* 46(6), 2016.
[28] G. E. Pibiri, R. Venturini. Clustered Elias-Fano indexes. *ACM TOIS* 36(1), 2017.
[29] D. Lemire, N. Kurz, C. Rupp. Stream VByte: faster byte-oriented integer compression. *Inf. Process. Lett.* 130, 2018.
[30] J. Mackenzie, A. Mallia, M. Petri, J. S. Culpepper, T. Suel. Compressing inverted indexes with recursive graph bisection: a reproducibility study. *ECIR*, 2019.
[31] G. E. Pibiri, R. Venturini. Techniques for inverted index compression. *ACM Comput. Surv.* 53(6), 2021.
[32] E. D. Demaine, A. López-Ortiz, J. I. Munro. Adaptive set intersections, unions, and differences. *SODA*, 2000.
[33] J. Barbay, A. López-Ortiz, T. Lu. Faster adaptive set intersections for text searching. *WEA*, 2006.
[34] J. Barbay, A. López-Ortiz, T. Lu, A. Salinger. An experimental investigation of set intersection algorithms for text searching. *ACM JEA* 14, 2010.
[35] J. S. Culpepper, A. Moffat. Efficient set intersection for inverted indexing. *ACM TOIS* 29(1), 2010.
[36] B. Ding, A. C. König. Fast set intersection in memory. *PVLDB* 4(4), 2011.
[37] B. Schlegel, T. Willhalm, W. Lehner. Fast sorted-set intersection using SIMD instructions. *ADMS@VLDB*, 2011.
[38] H. Inoue, M. Ohara, K. Taura. Faster set intersection with SIMD instructions by reducing branch mispredictions. *PVLDB* 8(3), 2014.
[39] T. L. Veldhuizen. Leapfrog Triejoin: a simple, worst-case optimal join algorithm. *ICDT*, 2014.
[40] C. R. Aberger, A. Lamb, S. Tu, A. Nötzli, K. Olukotun, C. Ré. EmptyHeaded: a relational engine for graph processing. *SIGMOD*, 2016.
[41] S. Han, L. Zou, J. X. Yu. Speeding up set intersections in graph algorithms using SIMD instructions. *SIGMOD*, 2018.
[42] S. Kim, T. Lee, S.-w. Hwang, S. Elnikety. List intersection for web search: algorithms, cost models, and optimizations. *PVLDB* 12(1), 2018.
[43] J. Zhang, Y. Lu, D. G. Spampinato, F. Franchetti. FESIA: a fast and SIMD-efficient set intersection approach on modern CPUs. *ICDE*, 2020.
[44] G. Díez-Cañas. Faster-than-native alternatives for x86 VP2INTERSECT instructions. arXiv:2112.06342, 2021.
[45] C. Faloutsos, S. Christodoulakis. Signature files: an access method for documents and its analytical performance evaluation. *ACM TOIS* 2(4), 1984.
[46] J. Zobel, A. Moffat, K. Ramamohanarao. Inverted files versus signature files for text indexing. *ACM TODS* 23(4), 1998.
[47] K. Wu et al. FastBit: interactively searching massive data. *J. Phys.: Conf. Ser.* 180, 2009.

[48] S. Melnik, A. Gubarev, J. J. Long, G. Romer, S. Shivakumar, M. Tolton, T. Vassilakis. Dremel: interactive analysis of web-scale datasets. *PVLDB* 3(1), 2010.

[49] M. Curtiss et al. Unicorn: a system for searching the social graph. *PVLDB* 6(11), 2013.

[50] Y. Li, J. M. Patel. BitWeaving: fast scans for main memory data processing. *SIGMOD*, 2013.

[51] L. Sidirourgos, M. Kersten. Column imprints: a secondary index structure. *SIGMOD*, 2013.

[52] F. Yang, E. Tschetter, X. Léauté, N. Ray, G. Merlino, D. Ganguli. Druid: a real-time analytical data store. *SIGMOD*, 2014.

[53] Z. Feng, E. Lo, B. Kao, W. Xu. ByteSlice: pushing the envelop of main memory data processing with a new storage layout. *SIGMOD*, 2015.

[54] S. Chambi, D. Lemire, R. Godin, K. Boukhalfa, C. Allen, F. Yang. Optimizing Druid with Roaring bitmaps. *IDEAS*, 2016.

[55] H. Lang, T. Mühlbauer, F. Funke, P. Boncz, T. Neumann, A. Kemper. Data Blocks: hybrid OLTP and OLAP on compressed storage. *SIGMOD*, 2016.

[56] B. Goodwin, M. Hopcroft, D. Luu, A. Clemmer, M. Curmei, S. Elnikety, Y. He. BitFunnel: revisiting signatures for search. *SIGIR*, 2017.

[57] B. Hentschel, M. S. Kester, S. Idreos. Column Sketches: a scan accelerator for rapid and robust predicate evaluation. *SIGMOD*, 2018.

[58] J.-F. Im et al. Pinot: realtime OLAP for 530 million users. *SIGMOD*, 2018.

[59] B. Chattopadhyay et al. Procella: unifying serving and analytical data at YouTube. *PVLDB* 12(12), 2019.

[60] R. Schulze, T. Schreiber, I. Yatsishin, R. Dahimene, A. Milovidov. ClickHouse — lightning fast analytics for everyone. *PVLDB* 17(12), 2024.